

Deep Dive — Module 9: Apache PDFBox & Apache POI (File Parsing)

***Goal:** understand how uploaded binary files become plain text the pipeline can use — PDF extraction with PDFBox, DOCX extraction with POI, file-type routing (including the OCR fallback), and the chunking that follows extraction.*

Where it sits: These are the **front door of the data pipeline**. A user uploads `invoice.pdf` or `purchase_order.docx` — binary blobs the embedding model can't read. PDFBox and POI open those formats and hand back text, which is then chunked, embedded (Module 5), and stored in Qdrant (Module 2). If extraction yields nothing, OCR (Module 7) takes over.

1. Why parsers are needed

An uploaded file is raw bytes in a container format (PDF, DOCX). The semantic pipeline needs **plain text**. Each format has its own internal structure, so each needs a dedicated library to pull the readable content out. This is purely an *input* concern — get clean text, then the rest of the system treats every document the same way.

2. Apache PDFBox — digital PDF text

PDFBox loads a PDF and extracts its text with `PDFTextStripper`.

```
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.text.PDFTextStripper;

public String extractFromPDF(MultipartFile file) throws Exception {
    PDDocument pdf = PDDocument.load(file.getInputStream()); // load from upload stream
    String text = new PDFTextStripper().getText(pdf);          // extract all pages
    pdf.close();                                               // always release resources
    return text;
}
```

Key limitation: PDFBox extracts text only from **digital** (text-based) PDFs. A *scanned* PDF is just images of pages — PDFBox returns an empty string, which is the trigger to fall back to Tesseract OCR. Always `close()` the `PDDocument` (or use try-with-resources) to avoid memory/file-handle leaks under load.

3. Apache POI — DOCX text

POI reads Microsoft Office formats; for `.docx` the project uses `XWPFDocument` + `XWPFFWordExtractor`.

```
import org.apache.poi.xwpf.usermodel.XWPFDocument;
import org.apache.poi.xwpf.extractor.XWPFFWordExtractor;

public String extractFromDOCX(MultipartFile file) throws Exception {
    XWPFDocument docx = new XWPFDocument(file.getInputStream());
    String text = new XWPFFWordExtractor(docx).getText();
    docx.close();
    return text;
}
```

POI also handles `.xlsx` (via `XSSFWorkbook`) if spreadsheet ingestion is needed. Like PDFBox, it works **in-process** in the backend — no external service.

4. File-type detection & routing

A single entry point routes each upload to the right extractor, with OCR as the fallback path:

```

public String extractText(MultipartFile file) throws Exception {
    String name = file.getOriginalFilename().toLowerCase();

    if (name.endsWith(".pdf")) {
        String text = extractFromPDF(file);
        return text.trim().isEmpty() ? ocrService.extractText(file) : text; // OCR fallback
    } else if (name.endsWith(".docx")) {
        return extractFromDOCX(file);
    } else if (name.endsWith(".txt")) {
        return new String(file.getBytes(), StandardCharsets.UTF_8);
    } else if (name.matches(".*\\.(png|jpg|jpeg|tiff)$")) {
        return ocrService.extractText(file); // images → OCR
    } else {
        throw new UnsupportedFileTypeException("File type not supported");
    }
}

```

Security note: routing on the filename extension alone is unsafe — a renamed `.exe` could slip through. The project validates the **real MIME type** (Apache Tika sniffing) and scans for malware before parsing. The extension check above decides which parser to use; the MIME/AV checks decide whether to parse at all.

5. Chunking after extraction

Extracted text is split into **~800-character chunks**, each becoming one embedding and one Qdrant point.

```

public List<String> chunkText(String text) {
    List<String> chunks = new ArrayList<>();
    int chunkSize = 800;
    for (int i = 0; i < text.length(); i += chunkSize) {
        chunks.add(text.substring(i, Math.min(text.length(), i + chunkSize)));
    }
    return chunks;
}

// A 2,400-char invoice → 3 chunks; all 3 share the same document_id in their Qdrant payload

```

Why chunk at all? Embedding models represent roughly a paragraph well; feeding an entire long document into one vector blurs its meaning. Chunking keeps each vector focused, and grouping chunk results back by `document_id` (Module 2) reconstructs document-level

matches. The project uses 800 chars with **no overlap** (field positions in financial docs are stable) and caps very long documents at 50 chunks for search.

6. Where this fits the whole pipeline

```
upload → extractText() — PDFBox (digital PDF)
                        | POI (DOCX)
                        | raw read (TXT)
                        └ OCR (images / empty-text PDF)
    → clean → chunkText() → embed (Module 5) → upsert Qdrant (Module 2)
    → persist metadata + ocr_used flag (Module 1) → audit
```

The `ocr_used` flag records whether the text came from a parser or from OCR, which matters for confidence and auditing.

7. Common pitfalls

- **Not closing `PDDocument` / `XWPFDocument`** — file-handle and memory leaks under load; use try-with-resources.
 - **Assuming all PDFs have text** — scanned PDFs are images; handle the empty-string case with OCR.
 - **Trusting the file extension** — validate the real MIME type (Tika) and scan for malware before parsing.
 - **Loading the whole file into memory carelessly** — large files (up to 50 MB) need streaming and the size limit enforced.
 - **Chunk boundaries cutting mid-number** — acceptable here (no overlap) but be aware it can split a value across chunks; the model still captures context.
 - **Ignoring encoding** — read TXT as UTF-8 explicitly.
 - **No cap on chunk count** — a huge document could create thousands of points; the project caps at 50 for search.
-

8. Practice exercises

Add `pdfbox` and `poi-ooxml` as Maven dependencies.

1. Extract text from a digital PDF with PDFBox and print it.
 2. Run the same code on a *scanned* PDF and confirm you get an empty/whitespace string.
 3. Extract text from a `.docx` with POI.
 4. Write the `extractText` router for `.pdf`, `.docx`, `.txt`, and image types, with the OCR fallback wired in.
 5. Implement `chunkText(800)` and verify a 2,400-char string yields 3 chunks of 800.
 6. Convert all chunks of one document into Qdrant point payloads sharing one `document_id` and incrementing `chunk_index`.
 7. Add a guard that rejects files over 50 MB before parsing.
 8. (Stretch) Add MIME-type validation (Tika) so a renamed `.exe` masquerading as `.pdf` is rejected.
-

9. Self-check questions

- Why can't the system embed an uploaded file directly?
 - What does PDFBox return for a scanned PDF, and what happens next?
 - Why must you close `PDDocument` / `XWPFDocument` ?
 - Why is routing on file extension alone insecure, and what does the project add?
 - Why is text chunked before embedding, and why ~800 chars?
 - How do chunk-level vectors get reassembled into document-level results?
 - What does the `ocr_used` flag tell you later?
-

10. Glossary

- **Apache PDFBox** — Java library for PDF text extraction.
- **PDFTextStripper** — PDFBox class that pulls text from pages.
- **Apache POI** — Java library for Office formats (DOCX/XLSX).
- **XWPFWordExtractor** — POI class for DOCX text.

- **MIME type** — the real file type, independent of extension.
 - **Apache Tika** — content/MIME detection library.
 - **Chunk** — a fixed-size text slice that becomes one embedding.
 - **chunk_index** — position of a chunk within its document.
-

Navigation: [← Module 8 JWT](#) | [Index](#) | [Module 10 How It Connects →](#)